

Plan Tecnico - Agente de Pre-Autorizacion Quirurgica en Tiempo Real

Hackathon

hackIAthon - Viamatica / ADEN

Reto seleccionado

Reto 1: Agente de Pre-Autorizacion Quirurgica
en Tiempo Real

Enunciado original

Actualmente, un paciente debe esperar horas o dias para que el seguro autorice una cirugia. El reto es crear un agente que reciba el informe medico digital (Hospital) y la poliza del paciente (Aseguradora) en una base de datos de Notion. La IA debe analizar si el

procedimiento
esta cubierto, si cumple con los requisitos de
carencia y emitir una preaprobacion o solicitud
de documentos faltantes de forma
instantanea.

Enfoque del producto

Herramienta pensada para el lado de la
Aseguradora: automatizar la decision de
pre-autorizacion que hoy toma horas o dias,
usando IA para leer los documentos y decidir
al instante. Notion funciona unicamente como
base de datos final (almacen), no como
punto de carga.

1. Flujo end-to-end

1. El usuario ingresa a un formulario web y coloca su numero de cedula.
2. Sube dos documentos: la poliza y el informe medico (PDF o foto/imagen).
3. El backend valida ambos archivos antes de procesar (ver seccion 4).

4. Si ambos archivos son validos, se
convierten a imagen si vienen en PDF.
5. Las imagenes se codifican en base64 y se
envian a la IA (Groq + Qwen VL) junto con
un
prompt que:
 - Extrae los campos clave de cada
documento.
 - Cruza cobertura, exclusiones y periodo
de carencia.
 - Devuelve un veredicto estructurado
(JSON): preaprobado, documentos
faltantes o
rechazado, con la razon.
6. Antes de procesar como caso nuevo, se
revisa si es un posible duplicado (ver
seccion 5).
7. El resultado (campos extraidos + veredicto
+ razon + fecha) se guarda en Notion junto
con
los dos PDFs/imagenes originales como
respaldo.
8. El usuario puede volver a consultar su
historial de casos ingresando cedula +

numero de
poliza (ver seccion 6).

2. Origen de los datos

- El paciente/usuario sube ambos documentos en el mismo momento (poliza + informe medico).
 - No se usa una base de polizas precargada por la aseguradora; cada caso trae su propia evidencia documental.
 - Los PDFs/fotos se guardan como respaldo adjunto en Notion; los datos que la IA extrae de ellos se guardan ademas como campos estructurados (texto, numero, fecha) para que el agente y las consultas futuras trabajen rapido sin reabrir el PDF.
-

3. Campos a extraer

Del informe medico

- Nombre del paciente
- Procedimiento o cirugía solicitada
- Diagnostico
- Fecha del informe
- Medico o centro emisor
- Folio o numero de seguimiento del documento (si existe)

De la poliza

- Numero de poliza
- Nombre del asegurado
- Fecha de inicio de vigencia
- Procedimientos o categorías cubiertas
- Exclusiones
- Periodos de carencia por tipo de procedimiento

4. Validacion previa (antes de llamar a la IA)

- El archivo no debe estar vacío ni corrupto.

- Debe ser un PDF o imagen legible (no otro tipo de archivo).
 - Tamano dentro de un rango razonable (ni 0 bytes ni excesivo).
 - Si falla esta validacion, no se llama a la IA: se devuelve de inmediato un mensaje claro pidiendo volver a subir el archivo.
-

5. Logica de estados y deteccion de duplicados

Estados posibles

- Preaprobado: el procedimiento esta cubierto y cumple carencia.
- Documentos faltantes: aplica cuando:
 - Falta uno de los dos archivos completos (subio solo poliza o solo informe), o
 - El archivo esta pero le falta un dato clave (ej. poliza sin fecha de inicio), o
 - El documento subido no corresponde a lo esperado (ej. subio una factura en vez

de la
poliza).

- Rechazado: el procedimiento no esta cubierto o no cumple el periodo de carencia.

Deteccion de duplicados (antes de procesar como caso nuevo)

Orden de prioridad de señales:

1. Folio o numero de seguimiento del documento: si existe y coincide con uno ya guardado en Notion, es duplicado directo.
 2. Si no hay folio, se cruzan señales combinadas: fecha cercana + mismo medico/centro emisor + mismo procedimiento. Si coinciden todas, se trata como duplicado. Si cambia el medico o pasa suficiente tiempo, se trata como caso nuevo legitimo aunque el diagnostico se repita.
-

6. Consulta de historial

- Llave de acceso: cedula + numero de poliza (evita que alguien acceda al historial de otra persona adivinando solo la cedula).
 - Devuelve todos los casos anteriores de esa persona con su veredicto y fecha.
-

7. Arquitectura tecnica

- Frontend + backend: Next.js (un solo proyecto, usando API routes para el backend).
- Conversion de documentos:
 - Si es PDF: conversion a imagen por pagina en el backend (libreria tipo pdf-to-img o pdf.js).
 - Si ya es imagen (foto): se usa directamente, sin conversion.
 - Ambos caminos terminan en una imagen codificada en base64 (funcion

nativa de Node, sin libreria adicional).

- IA: Groq API, modelo Qwen VL (mejor desempeño en lectura de documentos densos/OCR que Llama Vision). Mismo modelo hace extraccion + analisis + veredicto en un solo llamado.
- Base de datos: Notion (API oficial + integracion interna). Guarda campos estructurados, PDFs/imagenes originales adjuntos, y el veredicto con razon y fecha.
- Hosting: Vercel (despliegue directo desde el repo de GitHub, detecta Next.js automaticamente, da la URL publica requerida como entregable).

Variables de entorno

- Claves reales (Groq, Notion) SOLO en variables de entorno de Vercel. Nunca en el codigo del repositorio.

- Archivo .env local + .gitignore para desarrollo local.
 - Se sube al repo un .env.example con los nombres de las variables, sin valores reales.
 - GitHub Secrets queda como opcional, solo si mas adelante se automatiza algo con GitHub Actions; no es necesario para que la demo funcione.
-

8. Prompt a la IA: reglas anti-alucinacion

- Usar solo informacion literal visible en las imagenes/documentos.
- Si un campo no es legible o no aparece, marcarlo como "no encontrado", nunca inventarlo o asumirlo.
- Pedir evidencia: que cite/transcriba la parte exacta del documento donde encontro cada dato.

- Forzar formato de salida fijo (JSON) con valores permitidos limitados (ej. estado solo puede ser: preaprobado / documentos_faltantes / rechazado).
 - Temperatura baja (cercana a 0) para respuestas deterministas, no creativas.
-

9. Manejo de errores al llamar a la IA (Groq)

- Limite de 2 intentos por llamada.
- Mapeo segun el codigo de error que devuelve Groq:
 - 429 (Too Many Requests / rate limit): esperar el tiempo indicado por Groq (retry-after) antes de reintentar.
 - 5xx (error de servidor): reintentar con una espera corta.
 - 4xx distintos a 429 (ej. 401 no autorizado, 400 solicitud invalida): no reintentar, es un

problema de configuracion o del
archivo, no se resuelve solo.

- Si se agotan los 2 intentos, mostrar al usuario un mensaje segun la causa real:
 - Rate limit -> "el sistema esta ocupado, intenta en unos segundos".
 - Solicitud invalida / archivo -> "revisa que el documento subido sea legible".
 - Error de servidor -> "hubo un problema temporal del servicio de IA".
 - El caso no se pierde: si falla, se guarda en Notion con un estado de error para dejar registro, en vez de descartarlo silenciosamente.
-
-

11. Seguridad

Rate limiting

- Limite de peticiones en el propio backend (no en el de Groq), por direccion IP, para evitar que alguien agote la cuota gratuita

de Groq mandando solicitudes excesivas al formulario.

Validacion de entrada (usuario -> backend)

- La cedula y demas campos del formulario deben validarse contra el formato esperado (ej. solo numeros) antes de usarse.
- Ningun texto libre del usuario se inserta directo en el JSON que se manda a Notion sin sanitizar/escapar, para evitar romper la estructura o inyectar contenido no deseado.

Riesgo de inyeccion de prompt via imagen

- Un documento subido (imagen o PDF) podria contener texto oculto con instrucciones dirigidas al modelo (ej. intentar que ignore las reglas y apruebe el caso sin cumplir cobertura). Esto se mitiga con las reglas anti-alucinacion del prompt (seccion 8) y,

sobre todo, con la validacion de salida descrita abajo.

Validacion de salida (respuesta de Groq -> antes de guardar en Notion)

1. Validacion de estructura: el JSON debe tener exactamente los campos esperados, con el tipo correcto en cada uno (texto, fecha, etc). Si falta o sobra un campo, se rechaza.
2. Validacion de valores permitidos: el campo de estado solo puede ser uno de los tres valores definidos (preaprobado / documentos_faltantes / rechazado). Cualquier otro valor se trata como respuesta invalida.
3. Verificacion cruzada de coherencia: el backend recalcula la decision con reglas propias y simples a partir de los datos ya extraidos (procedimiento vs. lista de cobertura, fecha de vigencia vs. carencia). Si el veredicto de la IA no coincide con lo que esas

reglas propias calculan, el caso se marca como sospechoso / para revision en vez de guardarse directo como valido. Esta capa es la mas importante: la decision final no depende solo de lo que la IA responde.

10. Entregables del hackathon

- Enlace publico del agente funcional (URL de Vercel).
- Enlace del repositorio en GitHub o GitLab (sin claves reales expuestas).
- Envio de ambos al correo:
hackiathon@viamatica.com